DETECTION ENGINEERING SERIES

PART 2

THREAD TELEMETRY DESYNC

What Your EDR Sees vs. What Is Actually Running

SUMMARY

In the previous section, we examined PEB manipulation and how it creates a desynchronization between kernel and user-mode telemetry. We also noted that this technique is rarely used on its own. In real attack scenarios, it can be chained with other techniques.

In this part, we take the analysis one step further. Instead of focusing only on process-level manipulation, we look at telemetry differences that appear at the thread level. When techniques such as PEB spoofing, thread context manipulation, LOLBin chaining, stack spoofing, and indirect syscalls are combined, an attacker can create multiple telemetry views for the same process that do not fully match each other.

FOCUS AREA	DETECTION TYPE
Thread / Process Telemetry	Kernel-User Desynchronization

Birkan KES

Detection Engineer · March 2026 · ACTIVE RESEARCH · v1.0

Continued from: "HOW RELIABLE ARE THE LOGS??" (Part 1, February 2026)

▸ INTRODUCTION

Where Part 1 Left Off

In the first part of this series we started with a simple question: how much can you trust your logs? POC tests showed that PEB (Process Environment Block) manipulation can create a clear inconsistency between the information the kernel records at process creation time and the information user-mode tools read during runtime. The certutil.exe scenario (where a LOLBin that appears harmless in Sysmon quietly downloads a payload in the background) demonstrated exactly how a single-source detection approach can fail.

PEB manipulation is only the starting point. The real issue is that different telemetry layers do not always reflect the same event in the same way. For this reason, we move our analysis from the process level to the thread level. Telemetry desynchronization at the thread context level becomes a much more complex problem, both in terms of detection and correlating events across different data sources.

The core question remains:

> If a thread's Win32StartAddress, RIP, call stack and memory origin do not validate each other,
> **Which one does the EDR trust?**

In this paper we look at two attack scenarios. Scenario 1 combines PEB spoofing, thread desync and a LOLBin execution chain to create an inconsistent process view across different telemetry layers. Scenario 2 uses stack spoofing together with indirect syscall to bypass EDR detections that rely on user-mode hooks.

▸ FRAMEWORK

The 6-Layer Telemetry Model

Security telemetry for a process does not come from a single source. Different layers of the operating system produce data at different points in the process lifecycle. As a result, multiple telemetry views can exist for the same process. This is not a flaw; it is a design choice in Windows architecture. The problem appears when defenders treat all layers as equal or when attackers exploit the gaps between them.

Layer	Captured Data	Timing	Manipulation Risk
L0 Process Metadata	Sysmon Event 1 — kernel snapshot	Process creation time	Low — kernel recorded
L1 Thread Start Address	ETHREAD.Win32StartAddress	Thread creation time	Medium — stale after SetThreadContext
L2 Instruction Pointer	GetThreadContext → RIP (actual execution address)	Runtime	Low — live register state
L3 Call Stack	RtlVirtualUnwind stack frame chain	Runtime	High — user-writable stack
L4 Memory Origin	VirtualQuery.Type MEM_IMAGE — MEM_PRIVATE	Runtime	Medium — partial bypass possible
L5 Behavioral ETW	ETW Threat Intelligence Event 15 (SetThreadContext)	Event driven	Very Low — kernel event pipeline

Key Insight: The L1 ≠ L2 inconsistency can be observed in most cases. When a thread's Recorded Start Address (L1) differs from its actual instruction pointer (L2), this produces a measurable split between telemetry layers.

Stack spoofing (L3) and indirect syscalls (L4) can weaken some individual signals, however, these techniques cannot completely eliminate cross-layer inconsistencies.

L5 kernel ETW telemetry, on the other hand, observes the manipulation itself and is one of the hardest layers to suppress without kernel-level privileges.

► SCENARIO 1 · ATTACK TECHNIQUE

PEB Spoofing + Thread Desync + LOLBin Chain

Scenario 1 shows how chaining three techniques creates different telemetry views. Scenario 1 shows the telemetry divergence created by chaining three techniques. The process first presents a fake identity through PEB manipulation. Threads are created in a suspended state and redirected with SetThreadContext before execution begins (L1/L2 desynchronization). The actual execution then continues through a legitimate LOLBin. Depending on which telemetry layer is observed, the same process appears with three different identities at the same time.

3.1 PEB Identity Manipulation

`combine.exe` directly wrote into the `RTL_USER_PROCESS_PARAMETERS` structure inside its own PEB, changing both the `ImagePathName` and `CommandLine` fields to values pointing to `MicrosoftEdgeUpdate.exe`. In Part 1, only the CommandLine field had been manipulated; in this scenario, `ImagePath` is also modified, completing the identity spoofing at the user-mode layer. As a result, when tools such as Task Manager, Process Explorer, and Procmon read the PEB, they display a process identity that appears to be `MicrosoftEdgeUpdate.exe` instead of `combine.exe`

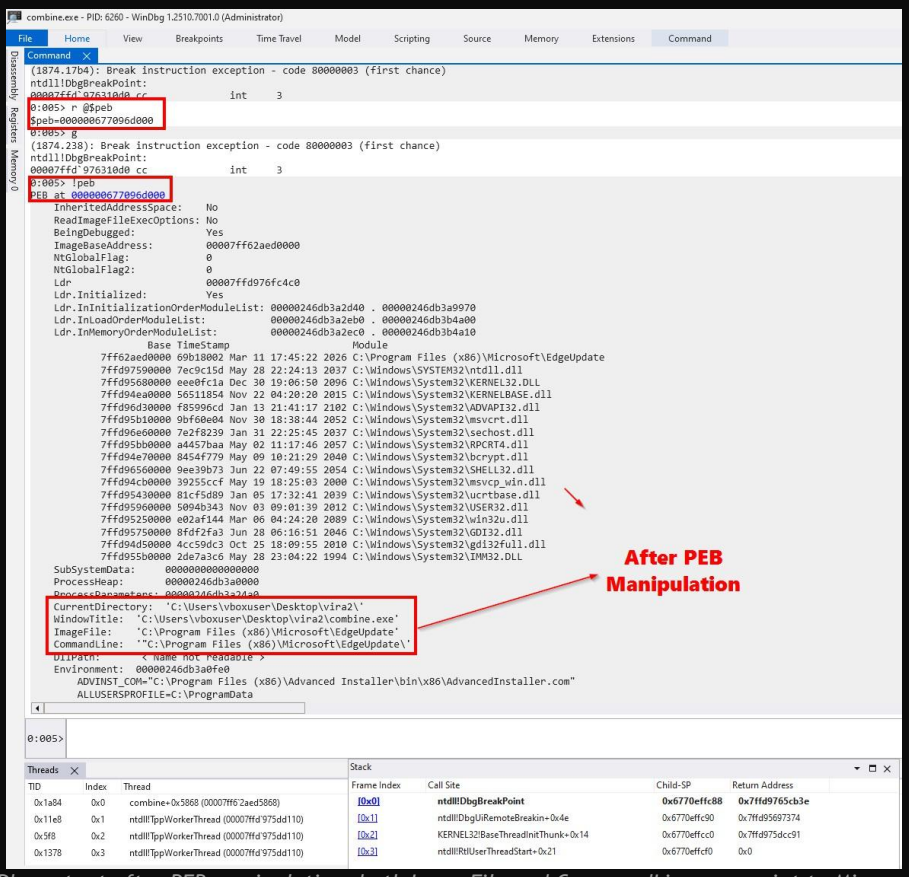

Figure 1 — WinDbg output after PEB manipulation: both ImageFile and CommandLine now point to MicrosoftEdgeUpdate. User-mode tools read the manipulated ImagePath as the real process identity. The kernel record captured at T-1 remains unchanged.

3.2 Procmon and Process Hacker: One Process, Two Identities

Running both tools on the same process at the same time clearly reveals the mechanism clear. Procmon collects data from two separate sources: the kernel event stream (PID 5376 = `combine.exe`, correct) and user-mode reads based on the PEB (ImagePath = `MicrosoftEdgeUpdate.exe` , manipulated).

When Procmon interprets this PEB data as a separate identity, it creates a second entry in the process list. The result is a "ghost process": PID 68 does not actually existed (no log record exists for it), it was simply generated from mismatched PEB reads.

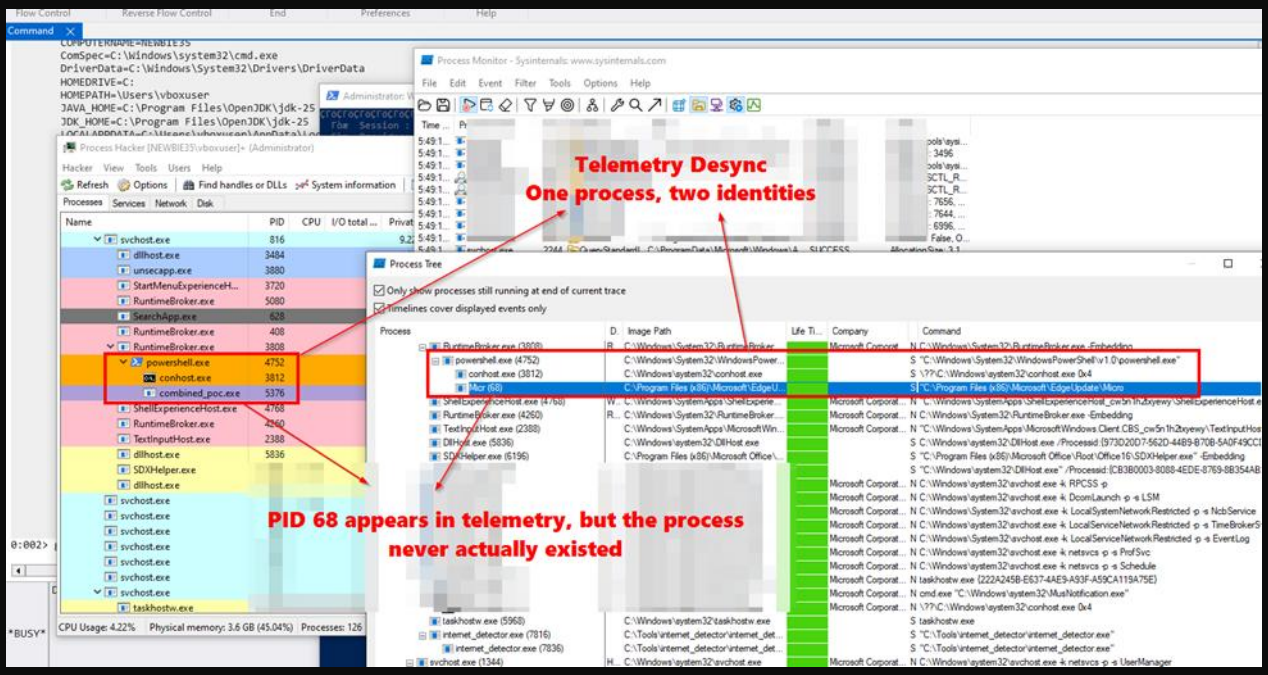

Figure 2 — Procmon (right) and Process Hacker (left): same process, two identities. PID 68 appears in Procmon telemetry but has no match in any kernel-level process list.

> ⚠ Key Finding: Procmon reads parent process chains from the PEB; which makes it not fully reliable for process tree analysis on its own.

3.3 Three Telemetry Sources, Three Different Stories

When `mshta.exe` was launched as the final stage of the execution chain, three different telemetry views were observed at the same time. Each tool correctly reported what it saw from its own telemetry layer.

However, these layers produced three different execution narratives for the same event, which do not align with each other:

Telemetry Source	What Does It Show?	Ground Truth
Telemetry 1 (Kernel / Sysmon)	combine.exe → mshta.exe execution chain	✓ Correct
Telemetry 2 (PEB View)	EdgeUpdate (PID 6296) → mshta.exe (PID 6296 never existed)	✕ Ghost
Telemetry 3 (Mixed Telemetry)	combine.exe → mshta.exe + child process EdgeUpdate (no kernel record for EdgeUpdate)	~ Misleading

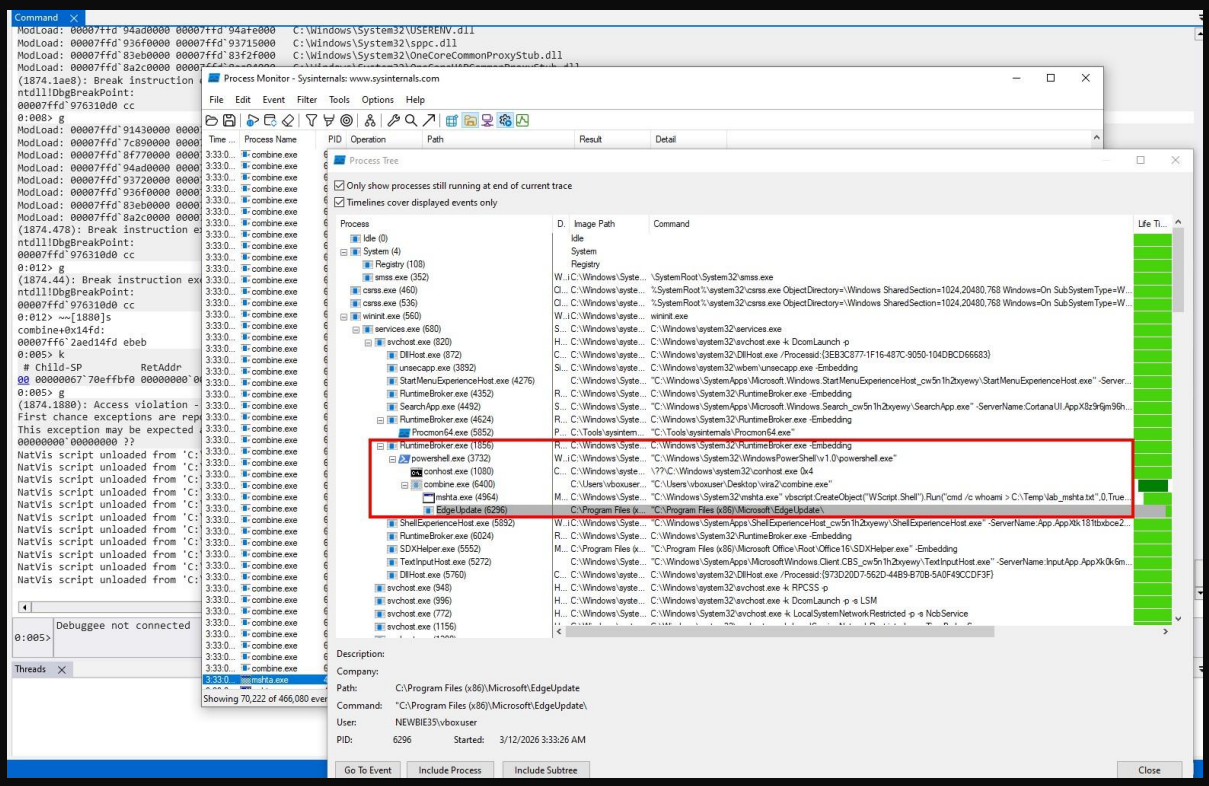

Figure 3 — The same execution chain appears as three different processes across telemetry layers.

You can only see this mismatch when all three telemetry views are looked at together. If you check each one alone, it may seem completely harmless.

► SCENARIO 2 · ATTACK TECHNIQUE

Stack Spoofing + Indirect Syscall

Scenario 2 targets the detection layers below process metadata. Instead of changing how a process appears, it changes how that process's API calls are observed by an EDR. The indirect syscall technique bypasses hooked ntdll functions and routes calls directly to the kernel. Stack spoofing manipulates the thread's call stack to look like a legitimate call chain. When used together, these techniques can partially bypass L3 (**Call Stack**) and L4 (**Memory Origin**) telemetry; but they leave a visible trace at L5, the **Behavioral** ETW telemetry layer.

4.1 Why Are EDR Hooks Targeted?

Today, detection platforms use kernel telemetry, `ETW` and memory inspection together. However, many behavioral signals still come from user-mode execution paths. That is why techniques like `indirect syscall` still matter in offensive research. In this approach, a small **`JMP patch`** is placed at the entry points of `NTAPI` functions inside `ntdll.dll`.
When a process calls a function like **`NtQueryVirtualMemory`**, **the call is first redirected to this hook. the EDR inspects the call's context and decides whether to let it continue.**

To bypass this mechanism, you need to skip the hooked function entirely and send the syscall directly to the kernel. Two things are required:

- The System Service Number (SSN) of the target function
- A valid stub or ret gadget that reaches the syscall instruction

4.2 FreshyCalls: SSN Resolution + Gadget Detection

The PoC parses the ntdll.dll export table, sorts all Nt* functions by virtual address and maps SSNs based on their position in the order. In the test environment, the SSN value for `NtQueryVirtualMemory` was found. The `0F 05 C3` (**syscall; ret**) gadget was found at `ntdll!NtAccessCheck+0x12`.

```
mov r10, rcx ; syscall calling convention
mov eax, 27h ; SSN = 39 → NtQueryVirtualMemory (build-specific)
mov r11, ntdll+0x12 ; syscall;ret gadget address (EDR inline hook bypassed)
jmp r11 ; execution transferred to ntdll MEM_IMAGE region
```

The critical property of the gadget address is that it resolves as `MEM_IMAGE + PAGE_EXECUTE_READ.` This means it resides in Microsoft-signed, mapped memory belonging to ntdll. When the syscall executes, the RIP points to this region. From the EDR's perspective, a memory-type check sees a legitimate, non-writable image page, rather than a `MEM_PRIVATE` or `RWX` region, which would normally trigger an alert. As a result, L4 is partially bypassed.

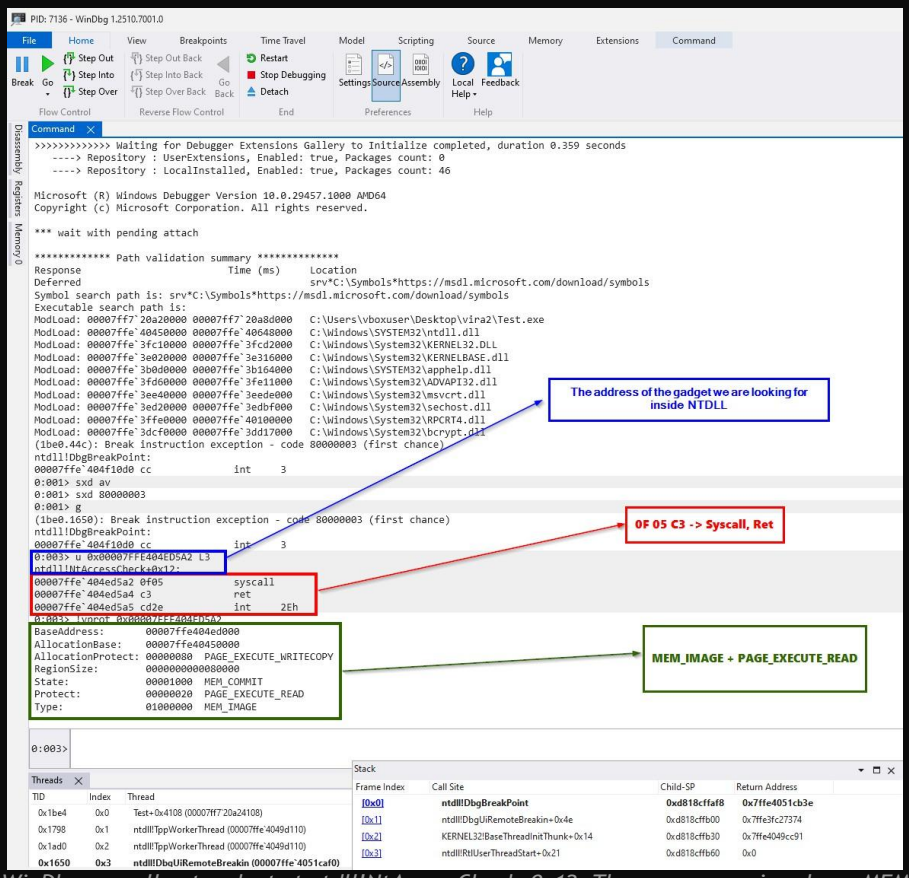

Figure 4 — WinDbg: syscall;ret gadget at ntdll!NtAccessCheck+0x12. The memory region shows MEM_IMAGE + PAGE_EXECUTE_READ; from the EDR's origin perspective this is completely legitimate (NTDLL) code.

Memory Type	Description	EDR Response
MEM_PRIVATE + RWX	Application-allocated, writable + executable — classic shellcode staging	⚡ ALARM
MEM_PRIVATE + RX	JIT-compiled code; suspicious if no known JIT engine is present	◻ INSPECT
MEM_IMAGE + RX	Mapped from a signed PE file — expected execution location	✓ ALLOW

4.3 Stack Spoofing: Fixing the Stack Depth Anomaly

Indirect syscalls solve the memory origin problem, but they create a new issue: the thread's call stack becomes too shallow. A legitimate API call normally produces a stack that traces back through **kernel32.dll** and can be followed up to RtlUserThreadStart. A direct jump to a gadget, however, leaves very few stack frames, which can trigger stack depth detection rules.

Stack spoofing addresses this by writing fake return addresses into the thread's stack (RSP is adjusted using SetThreadContext). n the PoC, BaseThreadInitThunk and RtlUserThreadStart were added as extra frames. As a result, the stack now passes the stack depth check; it contains three frames, both pointing to kernel32 and ntdll present, both pointing to kernel32 and ntdll. But the expected However, a critical frame kernel32!NtQueryVirtualMemorys still missing. This can be detected as an image_rop anomaly.

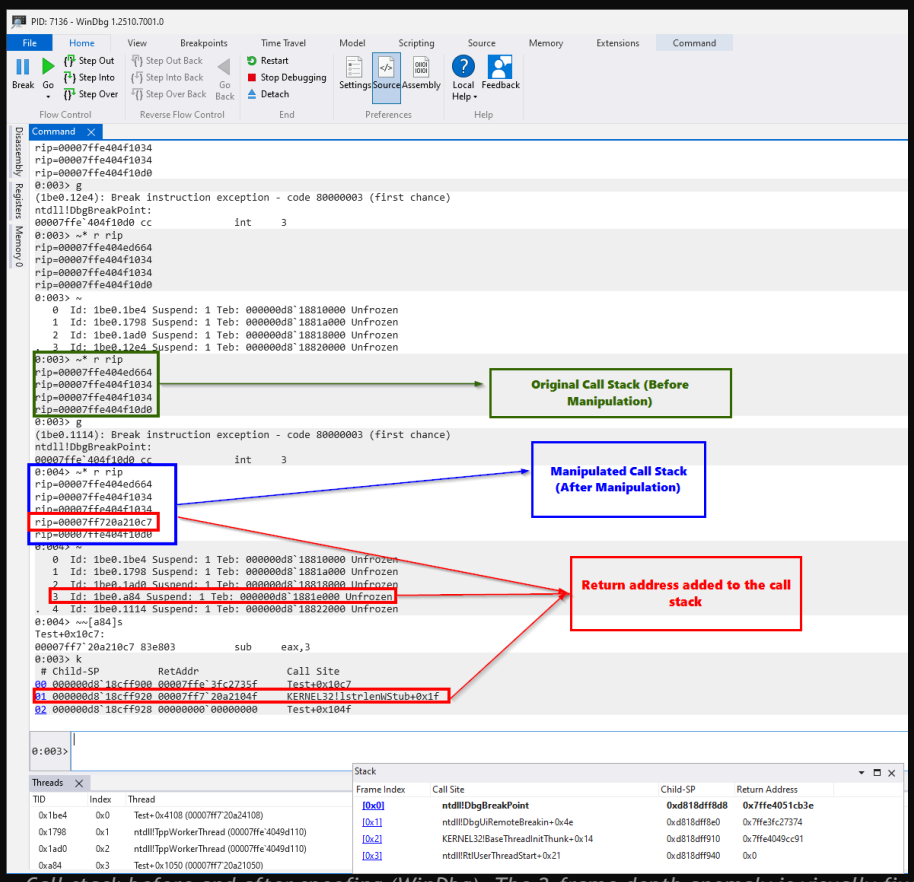

Figure 5 — Call stack before and after spoofing (WinDbg). The 2-frame depth anomaly is visually fixed, but kernel32!NtQueryVirtualMemory is missing from the call chain and is detectable as an image_rop anomaly.

4.4 Stub: MEM_PRIVATE Remains Visible

The stub that manages the indirect syscall lives in `MEM_PRIVATE` memory with `RWX` permissions. Its content (`mov r10, rcx` → `mov eax, 27h` → `jmp ntdll_gadget`) is directly scannable. An EDR that scans process memory and inspects executable private pages would be able to detect this stub. For this reason the technique is not a full bypass, but rather a partial evasion.

In practice, `MEM_PRIVATE + RWX` detection usually does not generate an immediate alert by itself. Instead, it often acts as a secondary signal that triggers deeper analysis. Because the false positive rate can be high, many vendors treat this check not as a standalone blocking rule, but as an indicator that becomes meaningful when correlated with other signals

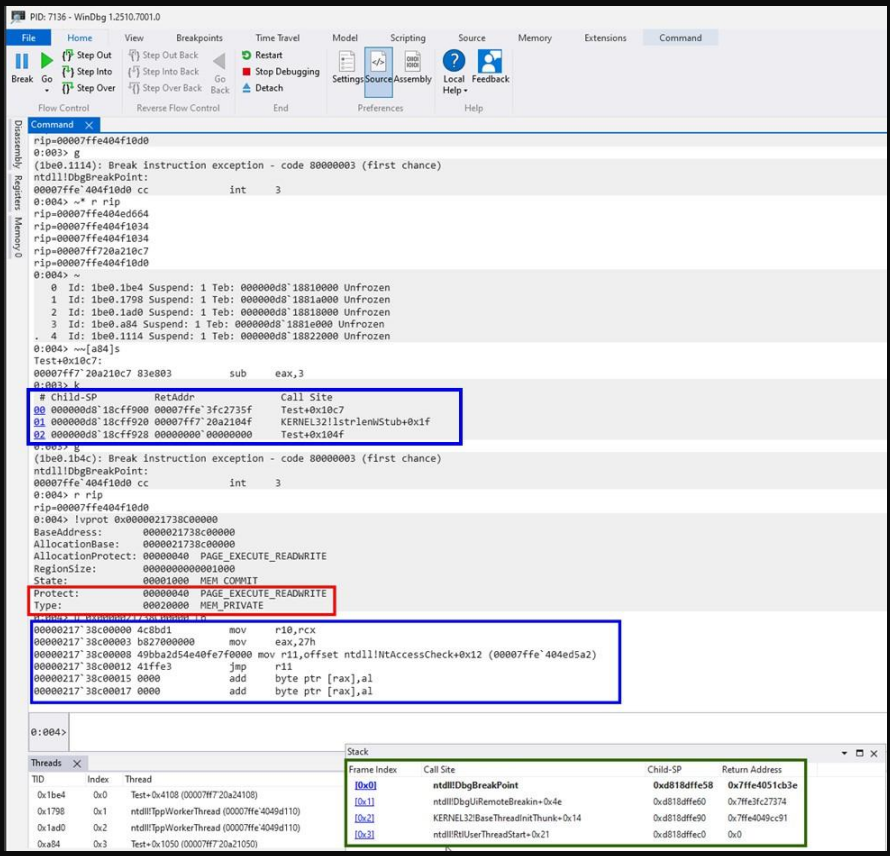

Figure 6 — Stub content read from WinDbg. The memory region shows MEM_PRIVATE + PAGE_EXECUTE_READWRITE; an EDR scanner can find and analyze this region directly.

> ✕ Partial Bypass: This technique combination does not fully bypass L4 (**Memory Origin**). Although indirect syscall execution shifts execution to a **MEM_IMAGE** region, the **MEM_PRIVATE** stub remains visible in memory.
>
> - At L3 (**Call Stack**), the missing **kernel32!NtQueryVirtualMemory** frame can be detected as an **image_rop** anomaly
> - At L5 (**Behavioral ETW**), each call to **SetThreadContext** triggers ETW TI Event 15, generating a detectable signal

▸ ANALYSIS

Scenario Comparison: Layer by Layer

The table below maps each telemetry layer against both scenarios, showing which layers were bypassed, which generate alerts and which remained structurally immune to manipulation.

Layer	Scenario 1 — PEB + Thread Desync	Scenario 2 — Stack Spoofing + Indirect Syscall	Bypassed?
L0 — Process Metadata (Sysmon Event 1)	UNCHANGED — kernel record immutable	UNCHANGED — kernel record fixed	NO
L1 — Thread Start Address (Win32StartAddress)	OUTDATED — set at thread creation, not updated after redirect	OUTDATED — same mechanism	PARTIAL
L2 — Instruction Pointer (GetThreadContext.Rip)	TRUTH — current execution address	TRUTH — current execution address	NO
L3 — Call Stack (RtlVirtualUnwind)	SHORT STACK anomaly — only 2 frames	FAKE STACK — looks normal but chain incomplete	PARTIAL
L4 — Memory Origin (VirtualQuery.Type)	MEM_IMAGE — normal	Gadget = MEM_IMAGE ✓ Stub = MEM_PRIVATE ✕	PARTIAL
L5 — Behavioral ETW (TI Event 15)	⚡ ALARM	⚡ ALARM	NO
PEB Identity — User-mode tools	SPOOFED — EdgeUpdate	—	YES (user-mode only)

Across both scenarios, the pattern is consistent: no single technique can bypass all six layers on its own. L0 (Process Metadata) and L2 (Instruction Pointer) are structurally immune; kernel records cannot be altered through user-mode manipulation.

L5 (Behavioral ETW) is triggered on every SetThreadContext call, regardless of the technique used. L3 (Call Stack) and L4 (Memory Origin) can be partially bypassed, but both still leave residual signals that a well-configured detection stack can identify.

> ⚠ Note: The Microsoft-Windows-Threat-Intelligence provider is off by default. In environments where it has not been enabled or collected by the SIEM, this signal never reaches the defender.

► DEFENSE GUIDE · BLUE TEAM

Minimum Detection Set

The following four signals are sufficient, together, to detect every technique demonstrated in this lab. None of them require a custom kernel driver. They can all be implemented using Sysmon, ETW, and a SIEM capable of cross-source correlation.

#	Signal	Source	What It Catches	Priority
1	ETW TI Event 15 (SetThreadContext)	ETW ThreatIntelligence provider	Catches every SetThreadContext call — both scenarios trigger here unconditionally	CRITICAL
2	L1/L2 Cross-Check (Win32StartAddress ≠ RIP)	NtQueryInformationThread + GetThreadContext	Thread desync — start address does not match the actual instruction pointer	HIGH
3	Sysmon Event 1 (Process Creation)	Sysmon kernel callback — immutable	Captures real Image, CommandLine, ParentImage — not affected by any PEB modification	HIGH
4	Abnormal stack depth + missing expected frames in CALL chain	Call stack analysis	Abnormal thread origin — finds spoofed frames even when stack depth looks normal	MEDIUM

Defense Architecture: Five Questions

Effective defense against these techniques requires asking five questions in order for every process of interest, rather than relying on a static rule list:

S

How did this process actually start?

PsCreateProcessNotifyRoutineEx
Immutable record — cannot be changed after creation.

↓

S

What were we told at that moment?

Sysmon Event 1
Compare CommandLine and Image fields against kernel callback data. Mismatch = ALARM

↓

S

What did the process do while it was running?

Sysmon Event 10 | API call telemetry | Injection signals
Behavior that diverges from the declared identity is a strong indicator.

↓

S

Does this behavior make sense in context?

Parent-child chain analysis | LOLBin calls
Time correlation between SetThreadContext events and network activity.

↓

S

Do all sources agree?

Kernel vs user-mode | Log vs memory | Event vs behavior
Discrepancy between layers is itself the signal.

▸ CONCLUSION

Logs Are Not Wrong. They Are Incomplete.

The techniques documented in Part 1 and Part 2 share a common design principle: they do not delete logs, crash EDR processes, or require a kernel exploit. What they do exploit is the fact that a defender observing a single telemetry layer may see something that is internally consistent, technically correct, yet fundamentally misleading.

The PEB contains mutable metadata relied on by user-mode tools. A thread's Win32StartAddress is written once and never updated. The call stack resides in user-mode writable memory. None of these are security vulnerabilities; the design decisions remain as intended, and the attacker's manipulations continue to function within those boundaries.

What changes is the complexity of exploiting these design decisions and the level of detection quality required to catch them.

> *The attacker's goal is not to hide. It is to disrupt perception to make malicious behavior appear as something the defender already trusts.*

Key Takeaways

- ✓ **L1 (Thread Start Address) ≠ L2 (Instruction Pointer)**
  L2 (instruction pointer) alone is not a reliable indicator for generating an alert; it is susceptible to runtime manipulation and can produce false positives. However, a discrepancy between L1 (Win32StartAddress) and L2 can be detected in all thread manipulation scenarios. No special tools are required; cross-layer correlation using existing kernel and user-mode telemetry is sufficient.
- ✓ **L5 (Behavioral ETW — TI Event 15) is the highest-reliability kernel signal**
  In the PoC, every SetThreadContext call triggered this event. If the Microsoft-Windows-Threat-Intelligence provider is correctly configured, thread hijacking cannot occur silently. This provider is off by default, so make sure it is enabled in your EDR and SIEM and that Event 15 is collected as a detection signal.
- ✓ **Stack spoofing (L3 — Call Stack) fixes stack depth but breaks CALL chain integrity**
  If pre-call return checks are applied, fake frames can be detected regardless of stack depth. Stack depth alone is not a sufficient detection criterion.
- ✓ **Correlation is required, not optional**
- ✓ No single tool can see the full picture alone. The attack only becomes visible when kernel and user-mode data are correlated. If your SIEM cannot do this automatically, closing this gap should be your first priority.

▸ REFERENCES

- Birkan KES — "How Much Can You Trust Your Logs? Kernel-User Telemetry Desync" (PART 1, February 2026)
- KlezVirus — "Eclipse: Detecting Stack Spoofing via CALL/Return Integrity" (2024)
- Alice Climent (RedOps) — "EDR Bypass via Indirect Syscall and Stack Spoofing" (2024)
- LetsMakeMalware — "FreshyCalls: Syscall Stub Extraction and SSN Resolution"
- Elastic Security Labs — "Kernel ETW is the Best ETW" (2024)
- Microsoft — PsSetCreateProcessNotifyRoutineEx2 / PS_CREATE_NOTIFY_INFO
- Microsoft — Windows Internals, 7th Edition: ETHREAD.Win32StartAddress behavior
- Trail of Bits — "ETW Internals for Security Research and Forensics" (2023)
- Velociraptor — "ETW Part 2: Process Parent Spoofing" (2021)